TUGAS PERTEMUAN 4

IMAGE ENHANCEMENT

Oleh :

KEVIN ANDHIKA

NIM 2311532005

MATA KULIAH IMAGE PROCESSING

DOSEN PENGAMPU AFDHAL DINILHAK, M.KOM.

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, MARET 2026

Tugas 1 : Resume Ringkasan Materi Pertemuan 4

Image Enhancement

Image enhancement adalah proses memperbaiki atau meningkatkan kualitas citra. Tujuan utamanya adalah mendapatkan citra dengan kualitas yang lebih baik dari citra awal sehingga interpretasi terhadap citra tersebut juga menjadi lebih baik. Penting untuk dipahami bahwa istilah "baik" di sini bersifat relatif, tergantung pada aplikasi dan masalah yang dihadapi.

Image enhancement merupakan salah satu proses awal dalam pengolahan citra. Kita perlu melakukan image enhancement karena beberapa alasan:

- Citra mengandung noise atau gangguan
- Citra terlalu terang atau terlalu gelap
- Citra kurang tajam atau kabur
- Terdapat cacat saat pengambilan citra
- Distorsi geometrik akibat lensa atau sudut pengambilan gambar

Berdasarkan domain operasinya, image enhancement dapat dikelompokkan menjadi dua kategori:

1. Image Enhancement dalam Domain Spasial

   Pada domain ini, kita melakukan manipulasi secara langsung terhadap pixel-pixel di dalam citra. T adalah operator yang bekerja pada citra input untuk menghasilkan citra output. Operator ini dapat bekerja pada satu pixel, sekelompok pixel bertetangga, atau keseluruhan pixel dalam citra.

2. Image Enhancement dalam Domain Frekuensi

   Pada domain ini, kita mengubah citra terlebih dahulu dari domain spasial ke domain frekuensi, baru kemudian memanipulasi nilai-nilai frekuensi tersebut. Masing-masing domain digunakan untuk tujuan spesifik karena tidak semua perbaikan citra dapat dilakukan dalam domain spasial.

Point processing adalah operasi yang dilakukan pada satu piksel saja, tanpa melibatkan piksel tetangganya. Contohnya meliputi:

- Thresholding: Mengubah citra menjadi hitam-putih atau biner berdasarkan nilai ambang batas (T).
- Image Negative: Membalikkan intensitas piksel (misal: 255 – f(x,y) untuk citra 8-bit).

- Image Contrast: Mengatur sebaran terang dan gelap. Kontras rendah berarti piksel berkumpul di satu area (terlalu gelap atau terlalu terang), sedangkan kontras baik berarti tersebar merata.
- Operasi Aritmetika: Penjumlahan/pengurangan/perkalian/pembagian antar citra atau dengan skalar untuk efek seperti pengurangan noise, pencerahan, atau penggelapan.
- Operasi Boolean: Operasi logika (AND, OR, NOT, XOR) yang penting untuk pemrosesan citra biner.
- Operasi Geometri: Seperti translasi (pergeseran), rotasi, dan penskalaan.

Image Histogram merupakan alat penting untuk memahami isi citra. Histogram adalah grafik yang menunjukkan frekuensi kemunculan setiap nilai intensitas piksel dalam suatu citra. Informasi penting dari histogram meliputi:

- Bentuk histogram menunjukkan kualitas citra:
    - Gelap (Underexposed): Piksel terkonsentrasi di sisi kiri (nilai rendah).
    - Terang (Overexposed): Piksel terkonsentrasi di sisi kanan (nilai tinggi).
    - Kontras Rendah: Histogram sempit dan terpusat di tengah.
    - Kontras Tinggi: Histogram tersebar merata dari kiri ke kanan.
- Normalisasi Histogram: Proses membagi frekuensi setiap nilai intensitas (n_i) dengan jumlah total piksel (n) sehingga nilai h_i (peluang kemunculan) berada di rentang 0 hingga 1 dan jumlahnya sama dengan 1. H_i ini juga dinotasikan sebagai $p(i)$, yaitu probabilitas sebuah piksel memiliki intensitas i.
- Nilai Rata-rata (Mean): $\mu = \Sigma k * p(k)$, menunjukkan rata-rata kecerahan citra.
- Variansi dan Standar Deviasi: Ukuran seberapa tersebar nilai intensitas dari nilai rata-rata. Semakin tinggi nilai variansi/standar deviasi, semakin tinggi kontras citra.

Histogram Equalization

Histogram equalization adalah teknik untuk mengubah distribusi nilai keabuan pada citra agar kontrasnya lebih menyebar pada seluruh rentang yang tersedia. Tujuan utamanya adalah meratakan distribusi intensitas pixel sehingga citra memiliki kontras yang lebih baik dan detail lebih mudah diamati.

Karakteristik Histogram Equalization

- Nilai keabuan yang sering muncul akan dijarangkan jaraknya dengan nilai keabuan sebelumnya. Ini berarti area dengan pixel padat akan diregangkan sehingga detail yang tadinya samar menjadi lebih terlihat. Misalnya, dalam foto wajah yang gelap, area wajah yang sebelumnya hampir hitam akan dibuat lebih bervariasi sehingga fitur wajah seperti mata dan hidung lebih jelas.

- Nilai keabuan yang jarang muncul bisa lebih dirapatkan jaraknya dengan nilai keabuan sebelumnya. Area dengan sedikit variasi pixel mungkin akan digabung atau didekatkan karena perubahan yang terjadi tidak terlalu signifikan.
- Histogram baru pasti mencapai nilai maksimal keabuan. Ini memastikan seluruh rentang intensitas dari paling gelap hingga paling terang digunakan secara optimal.

Histogram equalization bekerja dengan mentransformasi nilai intensitas lama ke nilai intensitas baru menggunakan distribusi probabilitas kumulatif. Prosesnya dimulai dengan menghitung histogram dari citra asli, kemudian menormalisasi histogram untuk mendapatkan probabilitas setiap intensitas. Selanjutnya, dihitung distribusi kumulatif dari probabilitas tersebut. Distribusi kumulatif inilah yang digunakan untuk memetakan nilai intensitas lama ke nilai baru.

Secara sederhana, pixel dengan intensitas yang sering muncul akan mendapatkan "ruang" lebih besar pada rentang intensitas baru, sementara pixel dengan intensitas jarang muncul akan mendapatkan ruang lebih kecil.

Setelah histogram equalization, citra akan memiliki histogram yang lebih merata. Bagian gelap menjadi lebih terang sehingga detail yang sebelumnya tersembunyi menjadi terlihat. Bagian terang mungkin sedikit digelapkan untuk menyeimbangkan distribusi. Hasilnya adalah citra dengan kontras yang meningkat dan detail yang lebih jelas.

Histogram equalization sangat berguna untuk meningkatkan kontras citra secara otomatis. Teknik ini membantu membuat detail citra lebih mudah diamati, terutama pada citra dengan pencahayaan buruk. Histogram equalization juga digunakan untuk standardisasi citra sebelum diproses lebih lanjut dalam suatu sistem.

Aplikasi histogram equalization sangat luas, mulai dari fotografi untuk memperbaiki foto yang kurang pencahayaan, pencitraan medis untuk melihat detail jaringan yang samar, hingga computer vision untuk mempersiapkan citra sebelum dianalisis oleh algoritma pengenalan pola.

Teknik ini menjadi salah satu alat fundamental dalam pengolahan citra karena kesederhanaan dan efektivitasnya dalam meningkatkan kualitas visual citra.

Tugas 2 : Program Image Enhancement Normalisasi Histogram

1. Mengambil Citra dari Github

   Kode ini mengambil dan mengimpor gambar dari repository GitHub. URL mentah dari gambar tersebut digunakan untuk mengunduh data, yang kemudian diubah menjadi objek gambar oleh pustaka PIL. Gambar ini akan menjadi bahan baku utama untuk semua proses selanjutnya.

```
image_url = "https://raw.githubusercontent.com/KevinAndT25/DIF62130_A_25_2311532005_KevinA

response = requests.get(image_url)
img = Image.open(BytesIO(response.content))

print("Original image loaded successfully.")
print(f"Image format: {img.format}")
print(f"Image size: {img.size}")
img
```

```
Original image loaded successfully.
Image format: JPEG
Image size: (636, 660)
```

2. Merubah ke Grayscale

Setelah gambar berhasil diambil, langkah berikutnya adalah mengubahnya dari gambar berwarna (RGB) menjadi gambar grayscale atau skala keabuan. Proses ini menyederhanakan data gambar dari tiga lapisan warna menjadi satu lapisan intensitas cahaya. Hasilnya adalah sebuah array dua dimensi di mana setiap angka mewakili tingkat kecerahan satu piksel, mulai dari 0 (hitam) hingga 255 (putih).

```
# Konversi ke grayscale
img_gray = img.convert('L')
img_array = np.array(img_gray)

print(f"Image converted to grayscale.")
print(f"Array shape: {img_array.shape}")
print(f"Data type: {img_array.dtype}")
print(f"Min value: {img_array.min()}, Max value: {img_array.max()}")
img_gray

Image converted to grayscale.
Array shape: (660, 636)
Data type: uint8
Min value: 12, Max value: 180
```

3. Proses Normalisasi Citra

Normalisasi histogram adalah proses mengubah data frekuensi absolut menjadi probabilitas. Pertama, kode menghitung histogram asli dengan menghitung jumlah piksel untuk setiap tingkat intensitas (0-255). Kemudian, setiap nilai frekuensi ini dibagi dengan total seluruh piksel dalam gambar. Hasilnya adalah histogram ternormalisasi yang menunjukkan probabilitas kemunculan setiap intensitas, bukan jumlah pastinya.

```
# Fungsi untuk menghitung histogram manual
def hitung_histogram(citra_array):
    """Menghitung histogram citra grayscale dengan rentang 0-255"""
    histogram = np.zeros(256, dtype=int)
    for baris in citra_array:
     for piksel in baris:
        histogram[piksel] += 1
        return histogram
```

```
# Fungsi untuk normalisasi histogram
def normalisasi_histogram(histogram, total_piksel):
    """Menormalisasi histogram dengan membagi setiap nilai dengan total piksel"""
    return histogram.astype(float) / total_piksel
```

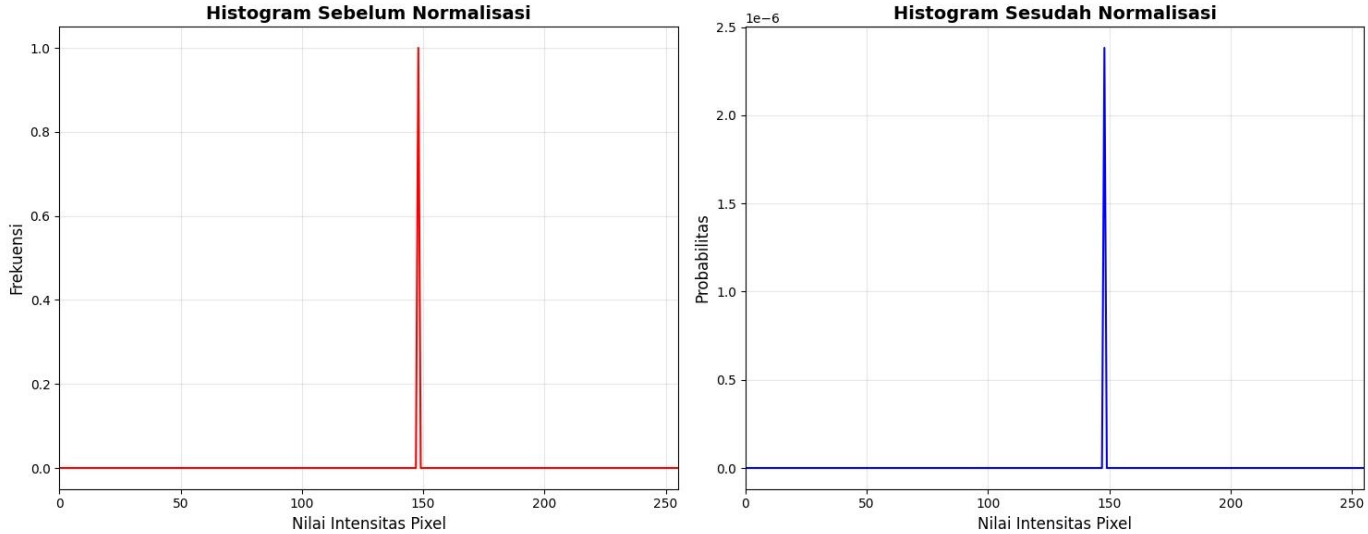
```
# Hitung histogram asli
hist_asli = hitung_histogram(img_array)
total_piksel = img_array.shape[0] * img_array.shape[1]
print(f"\nTotal piksel: {total_piksel}")
print("Histogram asli selesai dihitung.")

# Normalisasi histogram
hist_normalisasi = normalisasi_histogram(hist_asli, total_piksel)
print("Histogram ternormalisasi selesai dihitung.")
```

```
Total piksel: 419760
Histogram asli selesai dihitung.
Histogram ternormalisasi selesai dihitung.
```

4. Menampilkan Histogram Sebelum dan Sesudah Normalisasi

   Untuk memvisualisasikan hasilnya, kode menampilkan dua grafik histogram berdampingan. Grafik pertama menampilkan histogram asli dengan frekuensi absolut, sementara grafik kedua menampilkan histogram yang telah dinormalisasi dengan probabilitas. Perbandingan ini menunjukkan bahwa normalisasi hanya mengubah skala pada sumbu vertikal tanpa mengubah bentuk distribusi intensitas asli dari gambar.

```
plt.figure(figsize=(15, 6))

# Subplot 1: Histogram Sebelum Normalisasi
plt.subplot(1, 2, 1)
plt.plot(hist_asli, color='red', linewidth=1.5)
plt.title('Histogram Sebelum Normalisasi', fontsize=14, fontweight='bold')
plt.xlabel('Nilai Intensitas Pixel', fontsize=12)
plt.ylabel('Frekuensi', fontsize=12)
plt.grid(True, alpha=0.3)
plt.xlim(0, 255)

# Subplot 2: Histogram Sesudah Normalisasi
plt.subplot(1, 2, 2)
plt.plot(hist_normalisasi, color='blue', linewidth=1.5)
plt.title('Histogram Sesudah Normalisasi', fontsize=14, fontweight='bold')
plt.xlabel('Nilai Intensitas Pixel', fontsize=12)
plt.ylabel('Probabilitas', fontsize=12)
plt.grid(True, alpha=0.3)
plt.xlim(0, 255)

plt.tight_layout()
plt.show()
```



Tugas 3 : Program Image Enhancement Histogram Equalization

1. Proses Equalization Citra

   Proses equalization dimulai dengan menghitung fungsi distribusi kumulatif (CDF) dari histogram ternormalisasi. CDF ini kemudian digunakan untuk membuat tabel pencarian baru, di mana setiap intensitas asli dipetakan ke intensitas baru berdasarkan nilai kumulatifnya. Tabel ini lalu diterapkan ke setiap piksel dalam gambar asli untuk menghasilkan gambar baru dengan kontras yang lebih baik.

```
# Menghitung Cumulative Distribution Function dari histogram ternormalisasi
cdf = np.zeros(256, dtype=float)
cdf[0] = hist_normalisasi[0]
for i in range(1, 256):
    cdf[i] = cdf[i-1] + hist_normalisasi[i]
```

```
if cdf.max() < 0.999:
      print("Menormalisasi CDF...")
      cdf = cdf / cdf.max()
      print(f"CDF setelah normalisasi: {cdf.min():.4f} - {cdf.max():.4f}")
```

```
Menormalisasi CDF...
CDF setelah normalisasi: 0.0000 - 1.0000
```

```
# Membuat lookup table untuk transformasi intensitas
lookup_table = np.round(cdf * 255).astype(int)

# Mengaplikasikan lookup table ke citra
img_equalized = np.zeros_like(img_array)
for i in range(img_array.shape[0]):
 for j in range(img_array.shape[1]):
  intensitas_lama = img_array[i, j]
  intensitas_baru = lookup_table[intensitas_lama]
  img_equalized[i, j] = intensitas_baru
```

```
# Menghitung histogram citra hasil equalization
hist_equalized = np.zeros(256, dtype=int)
for i in range(img_array.shape[0]):
    for j in range(img_array.shape[1]):
            hist_equalized[img_equalized[i, j]] += 1
```

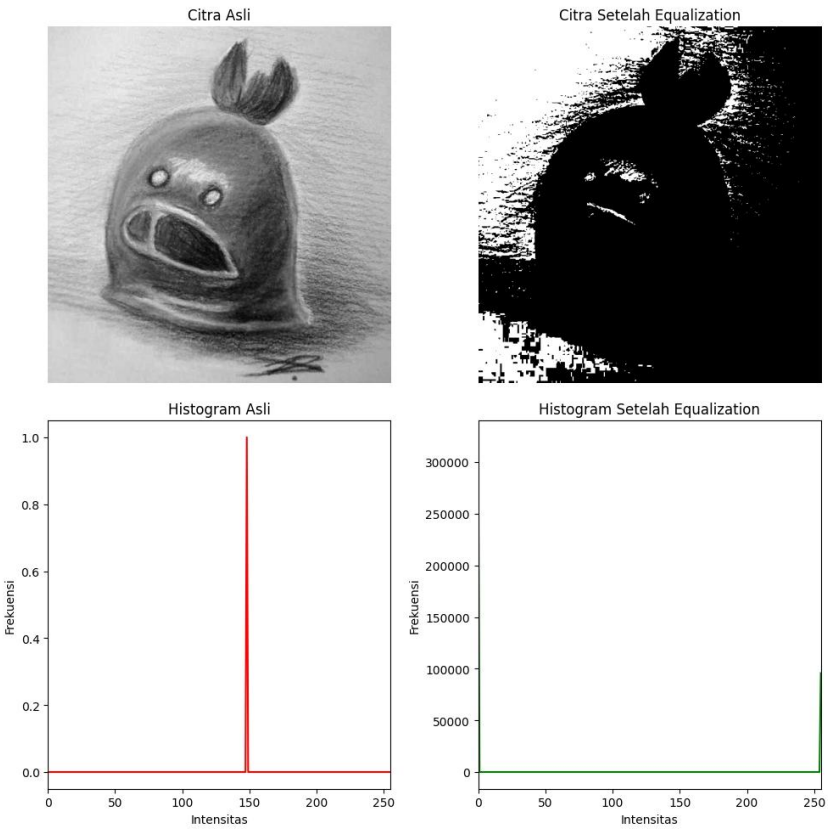
```
# Menormalisasi histogram hasil equalization
hist_equalized_norm = hist_equalized.astype(float) / total_piksel

print("Histogram equalization selesai")
print(f"Rentang intensitas baru: {img_equalized.min()} - {img_equalized.max()}")
```

```
Histogram equalization selesai
Rentang intensitas baru: 0 - 255
```

2. Menampilkan Perbandingan Citra Asli dengan Equalization

   Sebagai langkah akhir, kode menampilkan perbandingan visual dalam satu figur yang terdiri dari empat plot. Dua plot pertama menampilkan gambar asli dan gambar hasil equalization secara berdampingan. Dua plot berikutnya menampilkan histogram dari kedua gambar tersebut. Dengan melihat keempat plot ini, kita dapat secara langsung mengamati bagaimana pemerataan histogram berhasil menyebarkan distribusi intensitas dan secara visual meningkatkan kontras pada gambar.

```python
plt.figure(figsize=(15, 10))

# Baris 1: Citra
plt.subplot(2, 3, 1)
plt.imshow(img_array, cmap='gray')
plt.title('Citra Asli')
plt.axis('off')

plt.subplot(2, 3, 2)
plt.imshow(img_equalized, cmap='gray')
plt.title('Citra Setelah Equalization')
plt.axis('off')

# Baris 2: Histogram
plt.subplot(2, 3, 4)
plt.plot(hist_asli, color='red')
plt.title('Histogram Asli')
plt.xlabel('Intensitas')
plt.ylabel('Frekuensi')
plt.xlim(0, 255)

plt.subplot(2, 3, 5)
plt.plot(hist_equalized, color='green')
plt.title('Histogram Setelah Equalization')
plt.xlabel('Intensitas')
plt.ylabel('Frekuensi')
plt.xlim(0, 255)

plt.tight_layout()
plt.show()
```



Link Repository Github :
https://github.com/KevinAndT25/DIF62130_A_25_2311532005_KevinAndhika/blob/main/2311532005_IP-imageenhancement.ipynb